Research Report: After Acceptance: A Claim Graph for Residual-Machine Claims, Calibrated on eBPF

Author Name(s)
Affiliation(s)
Email address(es)

***Abstract*—Language-theoretic security asks whether a boundary recognizes the language that downstream machinery actually interprets. At a program-verifier boundary, an accepted artifact can subsequently drive stateful runtime words. We give a five-node claim graph that separates artifact acceptance, a same-suffix causal state distinction, bounded programmability, computed-report non-factorization, and a policy/threat obligation. Acceptance and causality induce an accepted-artifact-indexed causal language. Only after witnesses are jointly covered by an actual report cell does a word enter the report-relative residual language. A finite future-observation quotient and a factorization criterion make that obligation precise.**

We then state proof obligations for a uniformly controlled, observable, resettable gate and one fixed accepted interpreter that consumes an independently bounded circuit description. Prefix induction yields bounded composition when exact scheduling and frame conditions hold. An eBPF calibration case records A and C and supports P under explicit implementation premises. A dedicated, preallocated, non-LRU two-entry hash map implements NAND: after reset to one live sentinel, bits select existing- or fresh-key updates, and the second update's success predicate is the output. One fixed accepted program consumes canonical NAND DAGs with at most 64 inputs, 512 gates, and 578 live wires.

The latest source-snapshotted Linux/aarch64 run exercises named and random circuits, joint bounds, serial reuse, mechanism controls, and malformed descriptors. A separate author-run semantic auditor reconstructs descriptors and expected circuit semantics; a self-issued manifest provides a distinct integrity check. The case supports node P under explicit source-to-object, reset, frame, and serialization premises. It does not establish Linux report non-factorization or the policy/threat obligation. This separation is the report's main methodological result.

***Index Terms*—language-theoretic security, recognizers, residual languages, weird machines, program verification, abstract interpretation, eBPF**

I. Introduction

Language-theoretic security treats an input boundary as a language-recognition boundary: downstream processing is interpretation, and the input supplies the interpreted program [1]–[4]. Validation and program verification therefore share a recognizer-shaped role. Our question begins after acceptance. A verifier accepts a program artifact, but the accepted program can still drive a language of stateful helper and service operations. Which claims are needed before that downstream language can be called residual, programmable, shape-induced, or a weird machine?

The artifact language and the post-acceptance operation language have different carriers. A verifier can recognize bounded execution, typed helper use, and memory-access discipline without certifying the complete relation implemented through every documented runtime service. The accepted program may select operations, retain service state, observe returns, reset state, and compose effects. None of this alone implies a verifier defect. It first establishes a downstream operation language indexed by an accepted artifact.

The paper organizes the evidentiary burden as a claim graph:

Nodes C and R must not share one name. We call the accepted-artifact-indexed, same-suffix language L_{causal}. A word enters the report-relative residual language L^{R}_{res} only with an actual computed-cell collision. P and R branch after C: programmability does not imply report collision, and collision does not imply programmability. A policy-level weird machine requires P together with W; a *contract-shape-induced recognizer-relative* weird machine additionally requires R under documented semantics. The graph turns "weird machine" from a rhetorical label into checkable obligations.

Programmability is a separate constructive problem. A state distinction may be dead, uncontrolled, one-shot, or impossible to compose. We therefore require a program-visible readout, one uniform input dispatcher, a reset to a canonical class, and one fixed accepted interpreter with exact scheduling and frame preservation over an independently declared bounded descriptor domain. The resulting composition theorem is a proof-obligation decomposition, not a universal necessity theorem.

Our calibration case is Linux eBPF. A fixed artifact uses a dedicated two-entry hash map as a saturating resource. After reset, one sentinel is live. A zero bit updates the sentinel; a one bit inserts a fresh input-specific key. The second input-conditioned update succeeds except on input $(1, 1)$, yielding NAND. Ordinary bytecode still validates descriptors, selects keys, routes wires, controls the loop, and stores outputs. A host parser normalizes textual WMC1 into maps, while the verifier accepts only the fixed eBPF artifact.

The case is deliberately diagnostic. It establishes A and C and supports P under the stated implementation premises, even though the same function is easy to express in ordinary bytecode. It also shows why P cannot be silently promoted to R or combined with an absent W obligation. The retained verifier log is not an extractor for computed abstract cells, and the offline run supplies no violated deployment policy.

The paper contributes:

1) **A claim graph and typed languages.** We distinguish accepted artifacts, causal runtime words, report-relative residual words, bounded programmability, and

TABLE I
CLAIM GRAPH AND STATUS OF THE EBPF CALIBRATION CASE.

Node	Required fact	Status of the eBPF case
A	**Acceptance:** one fixed artifact belongs to L_V	recorded for the preserved object and environment
C	**Causal state distinction:** the same suffix and non-state context expose different observations from different selected runtime states	established for the second map update
P	**Bounded programmability:** accepted code uniformly controls, observes, resets, and composes the state under explicit frame and environment premises	source-level construction plus audited regression evidence
R	**Report-relative residual:** witnesses at C are jointly covered by an actual computed report cell that fails the behavioral factorization test	not established for Linux
W	**Policy/threat obligation:** an actor can drive the programmable behavior to an effect excluded by an intended policy	not established by the offline case

policy/threat obligations. A future-observation quotient and factorization criterion make node R testable, while countermodels show that the branches do not collapse.

2) **A bounded composition argument.** E1–E3 and the precise E4-D interpreter obligations isolate local gate correctness, reset, scheduling, and global frame conditions. Prefix induction proves the functional result; safety is a separate optional premise.
3) **An eBPF calibration case.** A saturating-rank NAND basis and fixed interpreter target $D_{64,512}$. The recorded suite covers named and fixed-seed random DAGs, deep and joint boundaries, a zero-gate case, serial reuse, mechanism controls, and malformed descriptors.

The empirical scope is one privileged, offline Linux/aarch64 environment. The case is not a Linux report-opacity result, concurrent deployment result, artifact-parametric compiler, unbounded machine, vulnerability, or policy-level weird machine.

II. RECOGNIZER-RELATIVE RESIDUAL LANGUAGES

A. Recognition, execution, and report

Let V be a recognizer over program artifacts and I the concrete execution semantics over concrete-state carrier Σ_I, including the instruction engine, helpers, stateful services, and relevant environment. The accepted artifact language is

$$L_V = \{P \mid V(P) = \mathsf{accept}\}. \tag{1}$$

For an optional concrete trace property Safe, the boundary is safety-sound when

$$\forall P \in L_V.\ \mathsf{Tr}_I(P) \subseteq \mathsf{Safe}. \tag{2}$$

Safety soundness is a premise, not an inference from one successful load. It is also distinct from completeness of an abstract transformer. Abstract interpretation relates concrete sets and abstract elements through abstraction and concretization maps [5], but a verifier's accepted language, its computed report, transfer soundness, and completeness are different judgments. In particular, the best abstraction of a singleton need not be the analyzer-computed cell at a joined control frontier.

When a report is in scope, let $\mathsf{Report}_V(P,\ell)$ be the finite set of abstract cells actually computed at frontier ℓ, with a declared concretization γ_ℓ. Frontier coverage requires

$$\mathsf{Reach}_I(P,\ell) \subseteq \bigcup_{a^\# \in \mathsf{Report}_V(P,\ell)} \gamma_\ell(a^\#). \tag{3}$$

Two concrete states are jointly covered only when one computed cell contains both. This point rules out a common but invalid shortcut: showing that two values have similar printed log text does not identify a computed abstract cell, a concretization, or joint coverage.

B. Causal words

For $P \in L_V$ and frontier ℓ, let $\Sigma_{\mathrm{op}}(P)$ be a finite alphabet of program-controllable runtime operations, and let $W^\ell_{\mathrm{run}}(P) \subseteq \Sigma_{\mathrm{op}}(P)^*$ be the words accepted code can execute from ℓ. Calling every such word residual would rename ordinary trace semantics, so node C uses a same-suffix causal test without yet asserting report omission.

Fix before comparison an observation contract

$$K_{\mathrm{obs}} = (\rho_{\mathrm{obs}}, \mathsf{Obs}, \mathsf{Slice}, \mathsf{Env}). \tag{4}$$

The projection ρ_{obs} selects the state candidate; Obs fixes the complete visible observation trace; Slice conservatively includes every non-selected component read by the suffix or observer; and Env fixes the same relevant resource configuration, schedule, nondeterminism, and external-interference choice for both executions. Write $\mathsf{ctx}_w(\sigma)$ for the suffix-read components other than $\rho_{\mathrm{obs}}(\sigma)$. The contract is sound for w only when equality of ctx_w fixes every non-selected value read by the suffix and observer, including clocks, registers, and service state; without that noninterference condition no C witness is claimed.

When defined, write $[\![w]\!]_I(\sigma) = (\tau, o, \sigma')$, where τ is the concrete suffix trace and Obs projects its declared visible observation.

Definition 1 (causal state-mediated word family). A word w is causal at (P,ℓ) when $P \in L_V$, $w \in W^\ell_{\mathrm{run}}(P)$, and there are $\sigma_0, \sigma_1 \in \mathsf{Reach}_I(P,\ell)$ for which $[\![w]\!]_I(\sigma_i) = (\tau_i, o_i, \sigma_i')$ are both defined and terminating in the same Env instance, and

$$\begin{aligned} \mathsf{ctx}_w(\sigma_0) &= \mathsf{ctx}_w(\sigma_1), \\ \rho_{\mathrm{obs}}(\sigma_0) &\neq \rho_{\mathrm{obs}}(\sigma_1), \\ \mathsf{Obs}(\tau_0) &\neq \mathsf{Obs}(\tau_1). \end{aligned} \tag{5}$$

Define the dependent tagged family

$$L_{\mathrm{causal}}(V, I; K_{\mathrm{obs}}) = \{(P, \ell, w) \mid w \text{ is causal at } (P, \ell)\}. \quad (6)$$

The artifact and frontier tags are part of the type. With fixed, decodable prefix encodings for artifacts, frontiers, and operation symbols, the family can be embedded in an ordinary language over one finite alphabet; the dependent notation keeps the carriers visible. A host descriptor is neither another element of L_V nor automatically a runtime word. It first becomes configuration, the accepted interpreter induces a schedule, and only a same-suffix witness enters L_{causal}.

The definition is observer- and slice-relative. Deleting an earlier input from the context after seeing the result would make the test circular. The eBPF case therefore declares its suffix from source semantics before comparison; the translated dump is retained for manual inspection rather than consumed by an automated slice checker.

C. Behavioral quotient and report-relative residual language

Fix a deterministic discipline

$$D = (S_D, A_D, O_D, \delta_D, \lambda_D, s_D) \quad (7)$$

with state carrier S_D, operation alphabet A_D, output alphabet O_D, and same-domain partial functions

$$\delta_D : S_D \times A_D \rightharpoonup S_D, \qquad \lambda_D : S_D \times A_D \rightharpoonup O_D. \quad (8)$$

The projection $s_D : \Sigma_I \to S_D$ is *adequate* for concrete semantics I when, for every concrete state σ and operation a admitted by D, a concrete a-step exists exactly when $\delta_D(s_D(\sigma), a)$ is defined; whenever $\sigma \xrightarrow{a/o}_I \sigma'$, then

$$\lambda_D(s_D(\sigma), a) = o, \qquad s_D(\sigma') = \delta_D(s_D(\sigma), a). \quad (9)$$

Here o is the complete declared step observation and Obs is a fixed projection of the resulting output word. Thus one projected state cannot hide different definedness or outputs. Every environment choice that affects a transition is fixed by D or included in S_D.

Let $\mathsf{Out}_D(r, w)$ be the partial complete output word obtained by iterating (δ_D, λ_D), write $\mathsf{Def}_D(r, w)$ for $\mathsf{Out}_D(r, w) \downarrow$, and take the continuation universe to be $\mathcal{W}_D = A_D^*$; infeasible words are represented by undefined output. For $r, r' \in S_D$, define

$$\begin{aligned} r \sim_D r' \quad &\Longleftrightarrow \quad \forall w \in \mathcal{W}_D. \\ &\Big[\mathsf{Def}_D(r, w) \Longleftrightarrow \mathsf{Def}_D(r', w)\Big] \\ \wedge \Big[\mathsf{Def}_D(r, w) &\Rightarrow \mathsf{Out}_D(r, w) = \mathsf{Out}_D(r', w)\Big]. \end{aligned} \quad (10)$$

Because $\mathcal{W}_D = A_D^*$ is closed under left-prefixing by an operation, $\sim_D$ is a right congruence: defined matching a-steps lead to equivalent successor states, since every successor continuation w is tested as aw. If $Q_D = S_D/\!\sim_D$ is finite, concrete execution induces a partial Mealy transducer over quotient states. This is a semantic quotient, not a verifier abstraction.

For a concrete set F, define the common enabled continuations

$$\mathcal{W}_D(F) = \{w \in \mathcal{W}_D \mid \forall \sigma \in F.\ \mathsf{Out}_D(s_D(\sigma), w) \downarrow\}. \quad (11)$$

Proposition 1 (behavioral factorization on a context fiber). Fix accepted P, frontier ℓ, and discipline D. Let $F \subseteq \mathsf{Reach}_I(P, \ell)$ agree on every non-selected context component read by the common continuation set $\mathcal{W}_D(F)$. Assume

$$\mathcal{W}_D(F) \subseteq W^{\ell}_{\mathrm{run}}(P) \quad (12)$$

Define

$$\beta_D(\sigma) = [s_D(\sigma)]_D, \qquad F_a = F \cap \gamma_\ell(a^\#). \quad (13)$$

The computed report has no cell-level behavioral collision on F if and only if

$$\forall a^\# \in \mathsf{Report}_V(P, \ell). \quad |\beta_D(F_a)| \le 1. \quad (14)$$

If the family of nonempty intersections $\{F \cap \gamma_\ell(a^\#) \mid a^\# \in \mathsf{Report}_V(P, \ell),\ F \cap \gamma_\ell(a^\#) \neq \varnothing\}$ partitions F, let $\pi_R : F \to \mathsf{Report}_V(P, \ell)$ map each state to its unique cell. The cardinality condition is then equivalent to $\beta_D = h \circ \pi_R$ on F for some $h : \pi_R(F) \to Q_D$.

Proof. A cell is collision-free exactly when all representatives it covers lie in one future-observation class. This is the cardinality condition and defines h on each nonempty cell. Under a partition, equal report labels imply equal quotient classes exactly when β_D factors through π_R. □

Definition 2 (report-relative residual language). Fix P, ℓ, D, F, and K_{obs} satisfying Proposition 1's hypotheses. A tagged word $(P, \ell, a^\#, w)$ is report-relative residual on this tuple when:

$$\begin{aligned} &(P, \ell, w) \in L_{\mathrm{causal}}, \qquad w \in \mathcal{W}_D(F), \\ &\sigma_0, \sigma_1 \in F \cap \gamma_\ell(a^\#) \text{ are Definition 1 witnesses}, \\ &a^\# \in \mathsf{Report}_V(P, \ell), \qquad \beta_D(\sigma_0) \neq \beta_D(\sigma_1). \end{aligned} \quad (15)$$

Write $\mathsf{Adm}(P, \ell, D, F; K_{\mathrm{obs}})$ when D is adequate and all context, common-continuation, and report-coverage hypotheses above hold. $L^R_{\mathrm{res}}(V, I, \mathsf{Report}; K_{\mathrm{obs}})$ is the union of these tagged words over tuples satisfying Adm. Because Obs projects Out_D, different observations under admitted w imply the displayed quotient-class inequality; writing it explicitly binds the definition to Proposition 1. This is a frontier collision, not a claim that a complete whole-program report can never recover the final function.

Instantiating Definition 2 requires an extractor for actual computed cells, a declared concretization, coverage, and a fixed context fiber. The retained Linux log does not provide these objects, so the eBPF case establishes L_{causal} but not L^R_{res}.

D. Shape, defect, and policy

A defect-induced gap uses behavior that violates the declared recognizer or runtime contract. A contract-shape-induced report gap instead uses documented semantics and preserves the declared safety contract while L^R_{res} is nonempty for a relation the report intends to certify. A policy-level weird machine additionally needs actor control and a security-relevant behavior excluded by an intended policy. An ordinary stateful API can therefore implement a transducer without satisfying either later classification.

III. Bounded State-Mediated Circuit Realization

A. From a distinction to a reusable gate

A causal word can expose one distinction without supporting repeated computation. Fix a deterministic gate discipline D, an adequate projection s_D, and one canonical reset class $q_0 \in Q_D$, identified with its subset of S_D. A gate basis $(P_G, reset, G, observe, D)$ satisfies:

E1 — causal basis and observation. At an internal frontier, two reachable, common-context states belong to different future-observation classes, and a common enabled suffix witnesses membership in L_{causal}. Accepted code stores or branches on the resulting bit.

E2 — uniform input control. One dispatcher G in accepted P_G reads runtime bits $x \in \{0,1\}^2$ and selects the complete gate word $u(x)$. For every admissible concrete state whose projection lies in q_0, and every environment allowed by D, execution is defined, terminates, and produces the same bit $g(x)$ for $g : \{0,1\}^2 \to \{0,1\}$. The experimenter is not an external oracle choosing a different constant program for each input.

E3 — reset. The reset word satisfies $reset \in A_D^*$. From every admissible concrete state σ, it is defined, preserves declared wire cells, and terminates in an admissible τ with $s_D(\tau) \in q_0$. If two pre-states agree on those wire cells and the fixed context and reset to τ_0, τ_1, then for every $x \in \{0,1\}^2$,

$$\mathsf{Out}_D(s_D(\tau_0), u(x)) = \mathsf{Out}_D(s_D(\tau_1), u(x)). \tag{16}$$

Without E1, a hidden difference has no program-visible effect. Without E2, the experimenter may supply the truth table. Without E3, the channel may be one-shot. A fourth condition below composes the gate without placing circuit semantics in an external oracle.

B. Descriptor domain

The artifact uses the bounded domain $D_{64,512}$. A descriptor is

$$d = (m, n, (s_i^0, s_i^1)_{0 \le i < n}), \tag{17}$$

with $0 \le m \le 64$, $0 \le n \le 512$, and

$$0 \le s_i^b < 2 + m + i \tag{18}$$

for each gate i and operand b. Write $m(d) = m$ and $n(d) = n$. Wire 0 is constant zero, wire 1 is constant one, primary inputs occupy wires 2 through $2+m-1$, and gate i writes canonical destination $2 + m + i$. Thus the maximum number of live canonical wires is $2 + 64 + 512 = 578$ and the highest valid wire index is 577.

For $x \in \{0,1\}^{m(d)}$ and gate function g, $\mathsf{Eval}_g(d, x)$ initializes the constants and input vector, then extends the wire vector in descriptor order:

$$\nu[2 + m + i] = g(\nu[s_i^0], \nu[s_i^1]). \tag{19}$$

The host encoding $\mathsf{Enc}_U(d, x)$ writes a normalized descriptor, inputs, and control record into maps. Textual WMC1 is a host serialization. Neither WMC1 nor d is a BPF program accepted by V; $\mathsf{Sched}_U(d, x)$ is the operation schedule induced when accepted P_U reads the encoding.

For $c = \mathsf{Enc}_U(d, x)$, let $\mathsf{PhysRun}_U(c) = (s, k, \mu)$ contain final status, completed-iteration count, and physical map state. Define the descriptor-relative canonical observation

$$\mathsf{WireObs}_d(\mu) = (\mu[0], \ldots, \mu[1 + m(d) + n(d)]) \tag{20}$$

and the status-masked interface

$$\mathsf{Run}_{U,d}(c) = \begin{cases} (s, \mathsf{WireObs}_d(\mu)), & s = \mathsf{OK}, \\ (s, \bot), & s \ne \mathsf{OK}. \end{cases} \tag{21}$$

The host may project requested output wires only from an OK result. This semantic rule does not assert that stale physical cells are erased.

E4-D — bounded data-parametric interpretation. One fixed artifact P_U discharges E4-D for $D_{64,512}$ when:

1) for the fixed map definitions and recorded load environment, $P_U \in L_V$ independently of descriptor contents d and x;
2) every valid $\mathsf{Enc}_U(d, x)$ establishes a pre-callback state $\mu^{(0)}$ with $\mu^{(0)}[0] = 0$, $\mu^{(0)}[1] = 1$, and $\mu^{(0)}[2 + j] = x_j$ for $0 \le j < m(d)$, executes exactly callback positions $0, \ldots, n(d) - 1$ in order, and terminates with $\mathsf{PhysRun}_U = (\mathsf{OK}, n(d), \mu)$;
3) one external critical section covers setup, invocation, and readback for every shared map in the footprint;
4) iteration i validates canonical form, reads exactly its two earlier source wires, resets and invokes the E1–E3 gate, writes the resulting g bit only to destination $2+m(d)+i$ and declared audit cells, and preserves the descriptor and all earlier wires; and
5) no execution performs more than 512 iterations, while malformed core-ABI controls or descriptor configurations covered by the declared validator terminate with non-OK status, at most 512 completed iterations, and semantic result $(s, \bot)$.

These clauses are deliberately proof obligations. In particular, E4-D requires the gate result to be written and exact iteration/terminal behavior; merely traversing a descriptor is insufficient.

C. Realization theorem

Theorem 1 (bounded composition under explicit obligations). Assume the fixed P_U discharges E4-D under its declared deterministic and serialized environment, and its embedded basis satisfies E1–E3 with function g. Then, for every $d \in D_{64,512}$ and $x \in \{0,1\}^{m(d)}$,

$$\mathsf{Run}_{U,d}(\mathsf{Enc}_U(d,x)) = (\mathsf{OK}, \mathsf{Eval}_g(d,x)) \tag{22}$$

after exactly $n(d)$ iterations. If the recognition boundary is additionally safety-sound for Safe, these traces also satisfy Safe.

Proof sketch. E4-D initializes the constant/input prefix. At iteration i, the descriptor bound makes both sources earlier cells. E3 supplies q_0, E2 supplies g from that class, and E4-D writes it only to the canonical destination while preserving the established prefix. Prefix induction yields Eval_g after $n(d)$ iterations; the exact-schedule clause supplies terminal OK. E1 establishes that the gate uses a causal state distinction but is not needed for the Boolean induction. Safety follows only from the separate soundness premise. □

The theorem decomposes local gate, reset, scheduling, and frame obligations; it is not a claim that tests enumerate the descriptor domain. It also does not prove E4-A, in which a compiler emits a different accepted BPF object per circuit.

IV. eBPF Calibration Case

A. Execution boundary

The program $P_U = wm\_circuit$ is one fixed eBPF artifact with section $SEC(\text{syscall})$. In the recorded environment the in-kernel verifier accepts it and userspace executes it offline through $bpf\_prog\_test\_run\_opts()$; Linux documents both userspace program execution and the syscall section/program-type mapping [6], [7]. No live hook is involved. The experiment is privileged and local, and its acceptance and resource facts remain specific to the recorded program type, kernel, and architecture.

The carrier boundary is:

The gate map $G0$ is a dedicated $BPF\_MAP\_TYPE\_HASH$ map with $max\_entries = 2$. It is non-LRU and retains the default preallocation; Linux documents the entry bound, default preallocation, update flags, and success/negative-error convention for this map type [8]. The discipline requires successful reset and one external critical section over setup, invocation, and readback for every map in the footprint. The serial harness satisfies the no-interleaving use pattern, but the eBPF program implements no concurrency lock.

B. Saturating-rank NAND

The gate uses pairwise-distinct keys: sentinel S and input keys A and B. Reset deletes all three and inserts S, establishing occupancy one. For the first input, zero updates S and one inserts fresh A; for the second, zero updates S and one inserts fresh B. Proposition 2 states the abstract occupancy law. For the Linux instance, existing-key success, below-capacity fresh-key success, and at-capacity fresh-key failure are explicit premises restricted to valid arguments, default preallocation, successful reset, no interference, and the recorded Linux 6.17 environment. Reference [8] supplies the map interface and return convention; retained raw returns and mechanism controls instantiate the law for this object. The gate observes only the second update's success predicate, not a portable error number.

The four executions are:

Proposition 2 (saturating-rank NAND). Let rank mean occupied-name count in a resource of capacity $k \geq 2$. Reset establishes rank $k-1$ with existing S, while pairwise-distinct A and B are fresh. Existing-name update succeeds without changing rank; fresh-name update succeeds and increments rank below k, and fails without changing rank at k. Dispatch zero to S, the first one to A, and the second one to B. The second update's success predicate is $\mathsf{NAND}(a,b)$.

Proof. When $a = 0$, the first update preserves rank $k-1$, so either second update succeeds. When $a = 1$, the first update raises the rank to k; the second update succeeds for $b = 0$ because S already exists and fails for $b = 1$ because B is fresh. The outputs are therefore $1, 1, 1, 0$. □

The eBPF instance uses $k = 2$. Compare inputs $(0,1)$ and $(1,1)$ immediately before the second operation. Both execute suffix $insB$ with the same key, value, map, flags, program point, observer, and recorded environment. Source and map semantics derive key sets $\{S\}$ and $\{S, A\}$; the run records success and failure. The earlier a is no longer read by the suffix, so its remaining suffix-relevant effect is the selected key-set/occupancy state. Thus $insB$ witnesses Definition 1 and L_{causal}.

Under the gate discipline, take $s_{D_G}(\sigma) = (phase(\sigma), K(\sigma))$, where phase ranges over the reset/sentinel/first/second-operation stages and $K \subseteq \{S, A, B\}$ with $|K| \leq 2$. The carrier, and hence its future-observation quotient, is finite. The complete reset-plus-gate word need not be causal because reset erases incoming differences; the witness is tagged by the internal frontier after the first operation.

This mechanism adds no extensional Boolean expressiveness to eBPF. It is a calibration case showing that a documented stateful service can supply a reusable post-acceptance gate; it does not by itself establish report omission.

C. Fixed interpreter and map ABI

In addition to $G0$, $wm\_circuit$ uses $TAPE$ and four interpreter maps:

- $TAPE$ records build variant, capacity, selected raw return, and error count;
- $CIRCUIT$ stores normalized records $(op, src0, src1, dst)$;
- $WIRES$ stores constants, primary inputs, and canonical SSA-style gate outputs;
- $VM\_CONTROL$ supplies the ABI version and declared counts;

TABLE II
CARRIER BOUNDARY IN THE EBPF CALIBRATION CASE.

Host data and validation	Recognition unit	Post-acceptance interpretation
textual WMC1 $\rightarrow$ normalized map configuration $\mathsf{Enc}_U(d,x)$	V accepts the fixed P_U, not WMC1 or d	P_U reads maps $\rightarrow \mathsf{Sched}_U(d,x) \rightarrow$ helper returns and wire observations

TABLE III
SATURATING-RANK NAND EXECUTIONS.

a	b	Operation word after reset	State before second operation	Second result	Output $[ret = 0]$
0	0	$updS; updS$	$\{S\}$	success	1
0	1	$updS; insB$	$\{S\}$	success	1
1	0	$insA; updS$	$\{S, A\}$	success	1
1	1	$insA; insB$	$\{S, A\}$	failure	0

- VM_TRACE stores per-gate validity, output, and, for state-mediated variants, the raw second-helper return.

The host parser checks and normalizes WMC1, writes map cells, and retains the requested output list. The list is projected only after $status = \mathsf{OK}$ and is never read by BPF. The source-level mask is guarded by host control flow; the negative suite tests rejection status and execution counts, not an injected runtime masking path. Inside the kernel, a bounded loop handles at most 512 descriptors. Each iteration copies descriptor and source values before helper calls, resets $G0$, invokes the gate, and updates the canonical destination by key, avoiding retention of map-value pointers across those calls.

Canonical destinations prevent source/destination aliases and forward references. Covered malformed versions, counts, opcodes, destinations, or sources receive non-OK status. For valid descriptors, the topological restriction makes every source earlier. External whole-transaction serialization prevents concurrent mixed configurations; successive serial invocations overwrite shared cells, and stale physical cells may remain.

The verifier's acceptance unit is the fixed interpreter artifact rather than each descriptor, although the harness may reload the same object for different datasets. Changing map configuration changes the interpreted circuit. This is the E4-D carrier distinction, not evidence for a compiler that emits a new accepted BPF object per circuit.

D. Source-level invariant and evidence boundary

Lemma 1 (source-level interpreter prefix invariant). Assume documented helper semantics, the E4-D serialization environment, valid $\mathsf{Enc}_U(d,x)$, and that the captured translated object preserves the manually inspected source control/data dependencies. After t successful callbacks, every canonical wire below $2 + m(d) + t$ equals the corresponding prefix of $\mathsf{Eval}_{\mathsf{NAND}}(d,x)$.

Proof sketch. The outer program initializes constants and preserves host-written inputs. A callback validates the current opcode, destination, and earlier sources, then copies the descriptor and source bits. Proposition 2 gives NAND after reset; success writes only the canonical destination and increments the completed count. The outer loop requests exactly $n(d)$ callbacks and checks both loop return and completed count. Induction on t gives the prefix claim. This is a source-level argument supported by retained translated dumps for manual inspection, not a machine-checked eBPF-semantics proof. □

The E4-D implementation argument maps to the artifact as follows:

Thus A and C are recorded, while P is supported under the stated source-to-object and serialization premises. Nodes R and W are absent: no Linux report-cell extractor or deployment policy is supplied.

V. EVALUATION

A. Recorded environment and primary run

The primary evidence is the report-bound source-snapshotted run `results/interpreter/interpreter-final-20260711-02/`. It records Ubuntu 24.04, Linux 6.17.0-35-generic on aarch64, and preserves four BPF variants, captured loaded-program metadata, verifier logs, descriptors, JSONL outputs, an author-run semantic audit, selected source including this report, and a self-issued SHA-256 manifest. It is an author-run reproduction of -06 on the recorded setup, not a third-party reproduction.

The dataset contains exactly

$$\begin{aligned} 38{,}533 = {} & 26{,}488 \text{ per-gate records} \\ & + 12{,}037 \text{ successful run records} \\ & + 8 \text{ negative-control records.} \end{aligned} \tag{23}$$

These are heterogeneous evidence rows, not "38,533 tests." In particular, the explicit arithmetic baseline's gate records do not contain state-mediated helper-return traces.

B. Coverage

Each named and random circuit exhausts its own finite input domain. The joint 64-input boundary necessarily uses two selected vectors rather than all 2^{64} inputs. The corpus is therefore regression evidence for the implementation and a check of the proof premises, not an enumeration or empirical proof of all $D_{64,512}$ descriptors.

The serial-alternation dataset reuses one loaded harness for 10,000 successive invocations. It detects common reset or

TABLE IV
E4-D OBLIGATIONS AND THE ARTIFACT EVIDENCE BOUNDARY.

Clause	Source/captured evidence	Empirical check or remaining premise
1 — fixed acceptance unit	preserved normal object, verifier metadata, captured loaded-program tag	descriptor changes do not create new BPF artifacts
2 — base and exact schedule	$wm\_circuit$ initializes constants, calls bpf_loop, and checks loop return/completed count	named, random, zero, deep, and joint-boundary runs
3 — serialization	runner invokes configurations serially; program contains no lock	whole-map-set critical section remains an external premise
4 — gate/frame	$circuit\_step\_cb$ validates/copies sources, resets $G0$, and updates one canonical destination	semantic auditor checks emitted destinations and outputs
5 — bounds/errors	count guards, callback failure status, host-side status-mask guard	eight negative cases check status/count; masking path is source-guarded only

TABLE V
RECORDED EVALUATION COVERAGE.

Dataset	Input coverage	Successful runs	Per-gate records	Purpose
9 named circuits	exhaustive per circuit	39	166	recognizable combinational functions, including NAND, mux, and adders
100 random DAGs	exhaustive per DAG; at most 6 inputs and 24 gates; seed 3235823838	1,876	23,776	fixed-seed structural and semantic regression
deep boundary	both valuations of one input to a 512-gate chain	2	1,024	gate-count and depth boundary
joint boundary	all-zero and all-one vectors	2	1,024	64 inputs, 512 gates, 578 wires, including wire 577
zero-gate boundary	dedicated repeat of the 0-input/0-gate constant descriptor	1	0	empty circuit execution
serial alternation	NAND/full-adder/mux alternation	10,000	0	reset and cross-invocation contamination regression
capacity-64 control	named-circuit inputs	39	166	removes capacity saturation
forced-sentinel control	named-circuit inputs	39	166	removes second fresh-key insertion
explicit baseline	named-circuit inputs	39	166	ordinary arithmetic NAND acceptance and semantics
malformed core	8 ABI/count/op/destination/reference cases	—	—	expected non-OK status and execution count

stale-state regressions under serial use. It is not a concurrent stress test and cannot establish the global critical-section premise; that premise remains external to the eBPF program.

C. Separate semantic audit and integrity check

The author-run semantic auditor does not trust the runner's pass flag. Its separate implementation:

1) reconstructs every named source descriptor and both generated boundary descriptors;
2) regenerates the 100-DAG corpus byte-for-byte from its fixed seed;
3) decodes WMC1, recomputes expected complete wire vectors, and compares each emitted gate destination and projected output;
4) checks every boundary-gate record and all eight malformed-core cases;
5) cross-checks each JSONL runtime tag against the captured loaded-program metadata.

The semantic audit reports success. After that audit, the run script writes and verifies a self-issued SHA-256 manifest as a separate integrity check; its verifier also reports success. The manifest detects accidental bundle divergence but is not strong provenance, a signature, timestamp, attestation, or independent reproduction. The captured tag is an anti-mix-up check, and the source snapshot covers only the harness's selected files.

D. Mechanism attribution

Three variants separate the capacity mechanism from the truth table and artifact acceptance.

First, increasing capacity from 2 to 64 removes saturation in the tested schedule; all 166 named-corpus control gate outputs become 1. Second, forcing the second one-bit input to update S removes the fresh-B insertion; again all 166 outputs become 1. Third, an arithmetic baseline computes NAND without the capacity mechanism and produces the expected named-circuit results. All four variants—the state-mediated gate, two controls, and baseline—load and execute in the recorded environment.

The controls support mechanism attribution: saturation and the second fresh name are both necessary for the gate's zero output. They do not show equivalent verifier reports. The baseline confirms that the mechanism gains no new Boolean function unavailable to ordinary bytecode.

E. Calibration result

Linux's verifier documentation states its objective in terms of determining program safety and validating paths, arguments, and memory access [9]; it does not specify a complete functional certificate for map-mediated computation. Leaving a helper return unknown may therefore be consistent with that documented objective. The case is a calibration, not evidence of unsound acceptance: it establishes node C and supports node P under its implementation premises, while the absent report extractor blocks node R.

F. Threats to validity

The experiment uses one kernel build and architecture. The argument relies on a dedicated preallocated non-LRU map, successful reset, distinct keys, the stated update laws, and whole-map-set mutual exclusion. Gate-emitting datasets preserve raw returns; the 10,000-run stress file records run/status/output evidence without per-gate raw rows. The proof uses only success versus failure and promises no portable error number.

Finite testing does not prove all $D_{64,512}$. Theorem 1 instead depends on the stated source-level initialization, validation, ordered iteration, gate, and frame obligations; the corpus seeks counterexamples. Translated dumps and verifier logs are retained for manual inspection and manifest hashing, not consumed by an automated data-flow checker. Status masking has a source-order guard, while the negative suite tests rejection/status rather than an injected masking path. There is no machine-checked eBPF-semantics proof, and production-verifier safety soundness is assumed only for the optional Safe conclusion.

VI. Related Work

Language-theoretic security supplies the recognition/interpretation framing [1]–[4]. Palmer, Rogers, and Adams reconstruct low-level interfaces as languages of buffers, calls, flags, references, and state-dependent operations [10]. Bratus et al. and constructive examples frame unintended computation as a programmable machine [11]–[13]; Dullien studies exploitability [14], and Paykin et al. make the source/target policy boundary explicit [15]. Vanegue's proof-carrying-code work is a close abstraction-side antecedent: computation outside a proof model can act as a shadow execution [16].

Abstract interpretation provides the vocabulary for concrete states, computed cells, soundness, and completeness [5], [17]. Our criterion concerns actual computed-cell factorization; it does not identify report uncertainty with standard abstract-interpretation incompleteness. PREVAIL is a separate eBPF analyzer with its own soundness argument [18], and tristate-number work proves domain-level properties [19], not end-to-end production-verifier soundness for this kernel. MOAT isolates BPF after acceptance [20], complementing our boundary analysis by limiting the effect of post-acceptance semantics. Anantharaman et al. locate weird instructions in mismorphisms across interpretations [21]; node R here requires the narrower witness that distinct future-observation classes occupy one actual computed report cell.

VII. Limitations, Implications, and Outlook

A. The claim graph has strict separations

Proposition 3 (non-implications among claim nodes). In general,

$$\begin{gathered} A \not\Rightarrow C, \quad C \not\Rightarrow P, \quad C \not\Rightarrow R, \quad P \not\Rightarrow R, \\ R \not\Rightarrow P, \quad P \not\Rightarrow W, \quad R \not\Rightarrow W. \end{gathered} \tag{24}$$

Moreover, a policy-level weird machine need not satisfy R; only the shape-induced recognizer-relative subclass requires $P \wedge R \wedge W$.

Proof by finite countermodels. For $A \not\Rightarrow C$, accept only *skip* and reject another safe constant program; the accepted system has no causal operation. For $C \not\Rightarrow P$ and $R \not\Rightarrow P$, use states $\{0, 1, \perp\}$, one read that outputs the bit and moves to sink $\perp$, no reset, and one report cell covering 0 and 1. For $C \not\Rightarrow R$, use a resettable two-state transducer whose report has one cell per future-observation class. For $P \not\Rightarrow R$, use any accepted bounded NAND interpreter satisfying E1–E4 and the same exact report partition. For $P \not\Rightarrow W$, let policy permit every output of that interpreter; using the one-cell destructive-read report with the same permissive policy also gives $R \not\Rightarrow W$. Finally, a defect-induced, policy-excluded machine can be fully described by a diagnostic report, so policy-level weird-machine status does not generally require R. □

Theorem 1 addresses only the C-to-P construction after its explicit reset and composition premises are supplied. Standard abstract-interpretation or acceptance incompleteness alone does not imply any later node.

B. Defensive implications

The model suggests a three-part audit. Declare the recognized property and report separately; enumerate the stateful operation language accepted code can drive, including environment assumptions; then test whether security-relevant quotient classes factor through the selected report. A contract violation calls for an implementation fix. Documented behavior that violates an intended report relation calls for report refinement, runtime restriction, or isolation. If the report never promised that relation and policy permits the behavior, there may be no verifier defect.

C. Weird-machine status and a future shape theorem

Under the source-to-object correspondence and serialization premises, the eBPF case supports node P: accepted code controls, resets, and composes a stateful NAND basis. It does not establish R or W. If documented causal behavior also fails a declared report-factorization contract, that additional gap is contract-shape-induced; the present Linux artifact does not establish this premise. Weird-machine security status further requires an actor and policy-excluded effect, absent from the offline run.

A future shape theorem must derive two closure properties rather than assume them. **Macro closure** must preserve acceptance when local operations are renamed and composed under a budget. **Report embedding** must preserve the relevant collision in actual computed cells after composition. Context-sensitive analysis can invalidate either property, and budgets need not compose additively. Complete-shell theory [17], instantiated with a declared observation semantics, may characterize required report refinement, but it does not create reachability, control, reset, routing, or policy violation.

TABLE VI
RELATION TO ADJACENT LINES OF WORK.

Line of work	Primary object	Separation added here
Language-theoretic security [1]–[4]	recognized input language and downstream interpreter	accepted artifact versus its post-acceptance operation words
Object-centric tracing [10]	effective low-level operation stream	accepted-artifact/frontier tags and a predeclared same-suffix causal test
Proof-carrying-code weird machines [16]	computation outside a proof abstraction	actual computed-cell factorization versus mere state-mediated programmability
Insecure compilation [15]	source/target contextual policy	report-relative node R separated from policy/threat node W

D. Remaining limitations

The evidence is one bounded combinational interpreter on one Linux/aarch64 setup. It is not a second-system, concurrency, E4-A compiler, or unbounded result. The report framework and calibration case remain disconnected at node R because no Linux report extractor exists. The final bundle is an author-run reproduction with a separate author-run semantic audit, not a third-party reproduction.

VIII. CONCLUSION

Program verification is a language-theoretic security boundary, but artifact acceptance and post-acceptance interpretation use different languages. We separated the accepted-indexed causal family L_{causal} from the report-relative L^{R}_{res}, gave a behavioral factorization criterion, and decomposed the obligations for bounded state-mediated composition.

The eBPF calibration establishes A and C and supports P under explicit source-to-object, environment, and frame premises. A two-entry map's saturating update algebra implements NAND, and one fixed artifact consumes circuits with at most 64 inputs, 512 gates, and 578 wires. It does not establish Linux report non-factorization or the policy/threat obligation. The claim graph makes that boundary a result rather than an ambiguity: post-acceptance programmability is measurable, but it is not yet a shape-induced recognizer-relative weird machine.

ETHICS, DATA, AND DISCLOSURE

All experiments run in an isolated local VM, attach no program to a live network or kernel hook, target no third party, and attempt no memory corruption, verifier bypass, or privilege escalation. The artifact uses legal helper calls and bounded execution.

The accompanying repository contains source, preserved BPF variants, environment records, verifier logs, translated disassembly, WMC1 descriptors, JSONL datasets, separate author-run audit code, and integrity manifests. The manifest is self-issued and protects only against accidental bundle drift.

If the target venue requires AI-use disclosure, the submission metadata should state that an AI-assisted workflow contributed to manuscript structuring and editing; the authors remain responsible for every claim, proof, citation, and artifact result. No conflicts of interest are declared.

REFERENCES

[1] L. Sassaman, M. L. Patterson, S. Bratus, and M. E. Locasto, "Security Applications of Formal Language Theory," *IEEE Systems Journal*, vol. 7, no. 3, pp. 489–500, 2013, doi: 10.1109/JSYST.2012.2222000.

[2] F. Momot, S. Bratus, S. M. Hallberg, and M. L. Patterson, in *2016 IEEE Cybersecurity Development (SecDev)*, pp. 45–52, 2016, doi: 10.1109/SecDev.2016.019.

[3] L. Sassaman, M. L. Patterson, and S. Bratus, "A Patch for Postel's Robustness Principle," *IEEE Security & Privacy*, vol. 10, no. 2, pp. 87–91, 2012, doi: 10.1109/MSP.2012.31.

[4] S. Ali, P. Anantharaman, Z. Lucas, and S. W. Smith, *IEEE Security & Privacy*, vol. 19, no. 3, pp. 17–23, 2021, doi: 10.1109/MSEC.2021.3059167.

[5] P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '77)*, pp. 238–252, 1977, doi: 10.1145/512950.512973.

[6] Linux Kernel Documentation, "Running BPF Programs from Userspace," [Online]. Available: https://www.kernel.org/doc/html/v6.17/bpf/bpf_prog_run.html (accessed Jul. 11, 2026).

[7] Linux Kernel Documentation, "Program Types and ELF Sections," [Online]. Available: https://www.kernel.org/doc/html/v6.17/bpf/libbpf/program_types.html (accessed Jul. 11, 2026).

[8] Linux Kernel Documentation, "BPF_MAP_TYPE_HASH, with PER-CPU and LRU Variants," [Online]. Available: https://www.kernel.org/doc/html/v6.17/bpf/map_hash.html (accessed Jul. 11, 2026).

[9] Linux Kernel Documentation, "eBPF Verifier," [Online]. Available: https://www.kernel.org/doc/html/v6.17/bpf/verifier.html (accessed Jul. 11, 2026).

[10] I. Palmer, E. Rogers, and R. Adams, "Object-centric Tracing for Language-Theoretic Security in Low-Level Interfaces," in *IEEE Security and Privacy Workshops*, 2026.

[11] S. Bratus, M. E. Locasto, M. L. Patterson, L. Sassaman, and A. Shubina, "Exploit Programming: From Buffer Overflows to Weird Machines and Theory of Computation," *USENIX ;login:*, vol. 36, no. 6, pp. 13–21, 2011.

[12] J. Bangert, S. Bratus, R. Shapiro, and S. W. Smith, "The Page-Fault Weird Machine: Lessons in Instruction-less Computation," in *7th USENIX Workshop on Offensive Technologies (WOOT 13)*, 2013.

[13] R. Shapiro, S. Bratus, and S. W. Smith, "'Weird Machines' in ELF: A Spotlight on the Underappreciated Metadata," in *7th USENIX Workshop on Offensive Technologies (WOOT 13)*, 2013.

[14] T. Dullien, "Weird Machines, Exploitability, and Provable Unexploitability," *IEEE Transactions on Emerging Topics in Computing*, vol. 8, no. 2, pp. 391–403, 2020, doi: 10.1109/TETC.2017.2785299.

[15] J. Paykin, E. Mertens, M. Tullsen, L. Maurer, B. Razet, A. Bakst, and S. Moore, "Weird Machines as Insecure Compilation," arXiv:1911.00157, 2019.

[16] J. Vanegue, "The Weird Machines in Proof-Carrying Code," in *2014 IEEE Security and Privacy Workshops*, pp. 209–213, 2014, doi: 10.1109/SPW.2014.37.

[17] R. Giacobazzi, F. Ranzato, and F. Scozzari, "Making Abstract Interpretations Complete," *Journal of the ACM*, vol. 47, no. 2, pp. 361–416, 2000, doi: 10.1145/333979.333989.

[18] E. Gershuni, N. Amit, A. Gurfinkel, N. Narodytska, J. A. Navas, N. Rinetzky, L. Ryzhyk, and M. Sagiv, "Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language De-*

sign and Implementation (PLDI '19), pp. 1069–1084, 2019, doi: 10.1145/3314221.3314590.

[19] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte, "Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers," in *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 254–265, 2022, doi: 10.1109/CGO53902.2022.9741267.

[20] H. Lu, S. Wang, Y. Wu, W. He, and F. Zhang, "MOAT: Towards Safe BPF Kernel Extension," in *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 1153–1170, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/lu-hongyi (accessed Jul. 11, 2026).

[21] P. Anantharaman, V. Kothari, J. P. Brady, I. R. Jenkins, S. Ali, M. C. Millian, R. Koppel, J. Blythe, S. Bratus, and S. W. Smith, "Mismorphism: The Heart of the Weird Machine," in *Security Protocols XXVII*, Lecture Notes in Computer Science, vol. 12287, pp. 113–124, Springer, 2020, doi: 10.1007/978-3-030-57043-9_11.